Building and Booting a Linux Kernel

By Kowshic Shankar R

Gathering the tools

- Install the tools mentioned below for building the kernel and booting it.

```
sudo apt update
sudo apt install -y git build-essential bc bison flex \
    libssl-dev libelf-dev libncurses-dev dwarves \
    cpio gzip qemu-system-x86 busybox-static
```

Linux kernel source

- The next step is to get the Linux kernel source.
- It can be found at kernel.org
- Or simply we can get it using git as below

```
git clone --depth 1 https://github.com/torvalds/linux.git ~/linux-minimal
```

Configurations

- The Linux kernel contains thousands of features and drivers. A particular machine does not need all of them.
- These features can be Installed, Packeged as Module or Disabled.
- For configuring there are several methods but as the features list is huge we can proceed with a default configuration.

```
make x86_64_defconfig
```

- make menuconfig is used as a graphical method.

Selecting features

- To make sure for the current example all needed features are enabled we use the below commands.

```
scripts/config --enable BLK_DEV_INITRD
scripts/config --enable BINFMT_ELF
scripts/config --enable SERIAL_8250
scripts/config --enable SERIAL_8250_CONSOLE
scripts/config --enable PROC_FS
scripts/config --enable SYSFS
scripts/config --enable DEVTMPFS
scripts/config --enable PRINTK
make olddefconfig
```

- The olddefconfig will update the future changes and saves the configuration

Compilation

- To compile the kernel we use the make command and compile a bzImage (Compressed binary Kernel Image).
- The make command takes a -j argument where we need to pass the no of processors used for compilation process.

```
make -j"$(nproc)" bzImage
```

- The nproc command returns the no of processors in the current system.
- After the process finished , you can find the compiled kernel at arch/x86/boot/bzImage

Root Filesystem

- We need a rootfs that will be packaged as the initramfs and used while booting.
- We need to create this structure with the busybox for shell and commands & init script.

```
mkdir -p ~/minimal-root/bin
mkdir -p ~/minimal-root/dev
mkdir -p ~/minimal-root/proc
mkdir -p ~/minimal-root/sys
```

```
minimal-root/
├── init
├── bin/
│   ├── busybox
│   └── sh -> busybox
├── dev/
├── proc/
└── sys/
```

Busybox

- Busybox is a single executable that contain basic commands and a shell.
- We need to include it with our filesystem to access commands like cat, mount etc and run the init.
- Also Busybox provides a basic shell called ash so it is linked to \bin\sh

```
cp /bin/busybox ~/minimal-root/bin/busybox
ln -s busybox ~/minimal-root/bin/sh
```

Init

- Create the Init file with the following contents that will be executed by the kernel as the first program.

  vi ~/minimal-root/init

```
#!/bin/sh
export PATH=/bin
busybox mount -t proc proc /proc
busybox mount -t sysfs sysfs /sys
busybox mount -t devtmpfs devtmpfs /dev
echo "Minimal Linux has booted!"
exec /bin/sh
```

initramfs

- We need to compress the contents in the rootfs into a package.

```
cd minimal-root
find . -print0 |
    cpio --null -o --format=newc |
    gzip -9 > ~/initramfs.cpio.gz
```

- The above command will compress all the contents of the folder to initramfs.cpio.gz

Booting

- Using qemu we will be booting the compiled linux kernel with the initramfs as the rootfs.

```
qemu-system-x86_64 \
    -m 512M \
    -kernel ~/linux-minimal/arch/x86/boot/bzImage \
    -initrd ~/initramfs.cpio.gz \
    -append "console=ttyS0 rdinit=/init" \
    -nographic
```

- The above command will boot into the ash shell

Booting

- Executing **uname -a** displays system and kernel information.
- **cat /proc/version** displays the kernel build version and compiler information.
- **ls /** displays the rootfs
- **mount** displays the currently mounted filesystems.

Key Learnings

- Compilation of Linux Kernel
- Enable/Module/Disable Linux Kernel Features (It include Device Drivers)
- Creation of initramfs and busybox
- Booting virtually using qemu